



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۲۰ آبان

فصل سوم: لایه *Transport* اسلاید ۸۹ تا ۱۱۳ [۱]

TCP congestion control

روش *AIMD = additive increase multiplicative decrease*

در این روش *tcp* به اندازه *cwnd* میتواند بسته *inflight* داشته باشد. افزایش اندازه آن در هر *RTT* به اندازه یک *MSS* بایت، موسوم به *Additive increase*. کاهش آن به صورت نصف شدن یا همان *multiplicative decrease* است. *cwnd* متغیر است. میتوان گفت:

$$lastByteSent - lastByteACKed \leq cwnd$$

و تقریباً میتوان گفت:

$$TCP\ rate = \frac{cwnd}{RTT} (bytes/sec)$$

slow start

در ابتدا *cwnd* برابر یک *MSS* بایت است و در هر *ACK* یک واحد *MSS* بایت به آن اضافه می‌شود و در هر *RTT* دو برابر می‌شود. سائز *cwnd* در هر *RTT* به ترتیب، ۱، ۲، ۴، ۸ و ... است. پس به صورت نمایی زیاد می‌شود. در هر *RTT* به اندازه *cwnd* تا سگمنت وجود دارد و به اندازه *cwnd* تا *ACK* دریافت کرده و اینگونه *RTT* دو برابر می‌شود. هر تعداد که *cwnd* باشد، به همان اندازه بسته ارسال میکنند.

congestion avoidance

فاز دوم است. در این جا به ازای هر *RTT* یک واحد اضافه می‌شود. یعنی به ازای هر *ACK* به اندازه $\frac{1}{cwnd}$

اولین بار که در مرحله *slow start* است، اول به صورت *TCP Tahoe* است و به صورت نمایی زیاد میشود، و اولین بار که به *sssthresh* رسید، از آن به بعد خطی اضافه می‌شود. مقدار اولیه *sssthresh* به صورت دیفالت تعیین شده. مثلاً ۶۴ کیلوبایت.

در هر فاز که باشیم، اگر اتلاف و *timeout* داشتیم، اول پنجره *cwnd* را نصف می‌کند، آن را در متغیر *sssthresh* قرار میدهم و بعد $cwnd = 1 \text{ MSS}$ میشود. و بعد به فاز *slow start* بر میگردیم تا بسته *miss* شده *retransmit* شود. مقدار *dupACKcount* هم با *timeout* صفر می‌شود.

اگر در *slow start* بودیم و $cwnd \geq sssthresh$ شد، به فاز *congestion avoidance* می‌رویم. چه در فاز *slow start* و چه *congestion avoidance* اگر ۳ تا *ACK* تکراری گرفتیم، که به حالت اتلاف ملایم معروف است، مقدار $sssthresh = cwnd/2$ و $cwnd = sssthresh + 3$ خواهیم داشت و وارد فاز *fast recovery* می‌رویم و باید آن بسته *miss* شده را *retransmit* کنیم.

در این فاز هم به ازای هر *ACK* تکراری (نه *ACK* جدید)، یک واحد *MSS* بایت به *cwnd* اضافه می‌شود. (یعنی میتوانیم در صورت امکان و جا داشتن پنجره، به بسته جدید، ارسال کنیم. چون به بسته قطعاً به گیرنده رسیده و بافر بدون خالی شده، پس میتونیم یکی جدید بفرستیم انگار)

در فاز *fast recovery* به ازای یک *ACK* جدید، مقدار $cwnd = sssthresh$ میشود و به فاز *congestion avoidance* برمیگردیم.

در هر فاز، در صورت دریافت *newACK*، مقدار $dupACKcount = 0$ میشود. و در صورت دریافت *ACK* تکراری نه جدید، یک واحد به $dupACKcount$ اضافه می‌شود.

throughput یا گذردهی *tcp* در فاز *congestion avoidance* بدون دونه به پنجره در هر *RTT* اضافه می‌شود و خطی اضافه می‌شود و هر وقت به w سایز پنجره رسید و اتلاف رخ داد، اندازه پنجره نصف می‌شود. پس میانگین اندازه پنجره بین $w/2$ تا w است. پس میانگین میشه $3w/4$ پس داریم: $TCP \text{ throughput} = \frac{3w}{4RTT} (\text{bytes/sec})$ درواقع به دو شکل نرخ *TCP* میتواند اضافه شود.

- نوع اول: کلاسیک: یعنی به صورت خطی و بدون دونه اضافه بشه
- نوع دوم: *cubic*: یعنی به صورت تابع درجه ۳ اضافه شود $(t - k)^3 + W_{max}$ که k در واقع زمان تقریبی رسیدن به W_{max} است. مثلاً $t_2 - t_1$ یا $t_3 - t_2$ و t مقدار زمان طی شده از اتلاف آخر است.)

درواقع دو فاز دارد. در فاز اول معروف به *steady state behaviour*. در ابتدا که پنجره نصف شده با شتاب بیشتر اضافه می‌شود و در حوالی W_{max} خیلی کم و نزدیک به صفر، زیاد میشود. تا دوباره نصف شود و همین روند تکرار شود. فاز دوم *max probing* یا پیدا کردن ماکسیسم است یعنی وقتی به W_{max} رسیدیم، دیگر *bottleneck* نیست. پس نصف نمی‌شود و همینجور t زیاد می‌شود. انقدر زیاد می‌شود تا W_{max} جدید را پیدا کنیم.

اتلاف موقعی رخ میدهد که به لینک *bottleneck* رسیده ایم و بسته‌ها از بافر روتر سرریز می‌شوند. معمولاً بافر پر

است و به ندرت خالی می‌شود. پس صف پر است معمولاً. پس علاوه بر تاخیر انتشار، تاخیر صف هم داریم. پس نباید نرخ *tcp* در مبدا را بیشتر از ظرفیت لوله *bottleneck* بیشتر نشود و بافر روترها حداکثر یک بسته در بافر خودشان داشته باشند.

بافر ها باید همیشه مقدار کمی بسته داشته باشند تا بتونن لینک جلوی خودشان رو تغذیه کنند. البته نه بیشتر اندازه بافر لوله: $bandwidth \times RTT_{min}$ است.

همیشه تعداد بایت های ارسالی در *RTT* آخر اندازه گیری و مدت زمان آن نیز محاسبه می‌شود که با *measured RTT* معروف است. و در نهایت $measured\ throughput = \frac{\#bytes\ sent\ in\ last\ RTT\ interval}{RTT_{measured}}$ *RTT* مینیموم نیز محاسبه می‌شود و به گونه ای برای حالت غیر ازدحام است. پس *throughput* ایده آل ما برابر $cwnd/RTT_{min}$ است.

اگر گذردهی *measured* نزدیک گذردهی ایده‌آل باشد، یعنی ازدحام کمی داشتیم و *cwnd* اضافه می‌شود ولی اگر خیلی کمتر بود یعنی ازدحام داریم و باید *cwnd* کاهش یابد.

اینجا *tcp* در حالت *delay – based* اینجوری نیست که با اتلاف، اون کنترل ازدحام *trigger* نمیشود. (برعکس حالت های *classis* و *cubic* که با اتلاف فعال میشدند) ثانیاً هدف اینه که فق لوله پر شه و نه بافر صف‌ها پس تاخیر را کوچک نگه میدارد تا *throughput* بیشینه می‌شود.

امروزه *tcp* های *delayed – based* استفاده می‌شود مثل $BBR = bottleneck\ bandwidth\ RTT$ است و توسط گوگل ارائه شده است.

در پروتکل های کنترل ازدحام قبلی شبکه، کمکی به کنترل ازدحام نمی‌گرد.

در پروتکل $ECN = explicit\ congestion\ notification$ روتر های سر راه وقوع ازدحام را خبر میدهند. یک نوع پروتکل *network – assistant* است.

دو بیت در فیلد نوع سریس در هدر *IP* وجود دارد به اسم *ECN*.

در *tcp* این امکان وجود دارد که *ecn* را فعال کرد.

دو بیت *CWR* و *ECE* در هدر *TCP* مربوط به آن است.

برای فعال سازی *ECN* باید آن را در *handshake* سه طرفه اولیه آن را فعال کنیم؟؟؟ در روتر دچار ازدحام، فیلد $ECN = 11$ میشود و وقتی به گیرنده برسد، فیلد $ece = 1$ را در *ACK* خود ۱ میکند. میشود. اگر به فرستنده برسد، اگر فرستنده اعتنا کند، سائز پنجره خودش فرستنده را کم میکند و با $cwr = 1$ به گیرنده میگوید انجام داده است.

TCP fairness

اینکه برای *K* اتصال، در دراز مدت سهم هر کدام از *bandwidth* برابر با R/K است.

خط گذردهی کامل و خط انصاف کامل اگر تقاطع پیدا کنند، نقطه تقاطع یعنی *fairness* برقرار است. *tcp* در شرایط ایده‌آل اگر تعداد اتصال ثابت باشند و همه در *congestion avoidance* باشند و همه اتصال‌ها یکسان باشد، انصاف وجود دارد و در غیر اینصورت ممکن است وجود نداشته باشد. مثلاً در مالتی مدیا، به خاطر موج دندان اره‌ای که ممکن است در *TCP* رخ میدهد با خاطر کنترل ازدحام آن، و نمیخواهند این را داشته باشند. پس با استفاده از *UDP* در وقت‌هایی که ازدحام در شبکه رخ میدهد، چون نرخ ارسال خود را در هنگام ازدحام کاهش نمیدهد اینگونه به تعداد یکسان اتصال *TCP* موازی باز کنند.

پروتکل $QUIC = \text{quick UDP internet connection}$

این پروتکل یک پروتکل کاربرد روی *UDP* است و با هدف افزایش کارایی *HTTP* است. تمام خدمات مثل کنترل ازدحام و کنترل خطا و برقراری اتصال را در لایه کاربرد پیاده‌سازی میکند. و در مرحله *handshake* اعمال مدیریت کلید رمزنگاری و تصدیق اصالت و با نگهداری حالت کنترل ازدحام و جریان را نگه میدارد و از تاخیر جلوگیری میکند. الگوریتم‌های *QUIC* شبیه الگوریتم‌های موجود در *TCP* عمل می‌کنند. درواقع *HTTP2(slimmed)* یا ساده شده روی *QUIC* با هم *HTTP3* را تشکیل می‌دهند. که آن هم روی *UDP* است. (در حالی که *http2* عادی روی *TLS* بود. که آن روی *TCP* بود) در *HTTP3* چندین *stream* سطح کاربرد میتوانند به طور موازی روی یه اتصال *QUIC*، مالتیپلکس و ارسال شوند. هر *stream* یک شیء *http* است مثلاً. (مشکل *http2* به قبل، *HOL blocking* بود) مشکل استاپ کردن کل ارسال برای *retransmit* کردن شیء خطا شده. در *UDP* و *HTTP3* با اینکه مثلاً جویبار میانی دچار خطا شود، بعدی‌ها ارسال می‌شوند.

قطع تماس در *TCP*

برای قطع تماس در *TCP*

مشتري اول با ارسال یک سگمنت با فلگ $FINbit = 1$ از حالت *established* به حالت *FINwait1* میرود. و دیگه نمیتونه داده ارسال کنه. سرور که گرفت، پیامش رو *ACK* یکند و به حالت *close - wait* میرود ولی میتواند هنوز داده ارسال کند.. کاربر با دریافت *ACK* به $2 - WAIT - FIN$ میرود. بعد از مدتی، سرور $FINbit = 1$ رو در یک سگمنت میفرستد و به حالت *LAST - ACK* میرود و دیگه نمیتونه داده ارسال کنه. کاربر بعد از دریافت آن، به حالت *timed - wait* میرود و یک سگمنت با $ACKbit = 1$ میفرستد. سرور بعد از دریافت آن به حالت *closed* میرود.

کاربر هم بعد از $2 \times \max \text{segment life}$ صبر کردن، به حالت *closed* می‌رود. که اون $\max \text{segment life}$ به طور تجربی حساب میشه.

رفتار *tcp* روی لوله های چاق: *fat pipes*

چون $\text{delay} - \text{bandwidth product}$ زیادی دارد (که نشان دهنده گنجایش داده لوله است) و لوله به عنوان بافر بزرگ عمل میکند.

گذردهی *tcp* برحسب احتمال اتلاف، به این شکل محاسبه می شود (طبق فرمول در مقاله ماتیس)

$$\text{tcp throughput} = \frac{1.22 \times MSS}{RTT\sqrt{L}}$$

که $L = \text{loss rate}$ مثلاً باید 2×10^{-10} باشد تا به گذردهی بالا مثلاً ۱۰ گیگابیت در ثانیه برسیم که فقط در فیبر نوری میتوان به این رسید. پس نیازمند پروتکل های *TCP* پیشرفته تر برای لینک های سریع و طولانی هستیم.

مراجع

[1] “ch3-v8-slide-89-to-113,” <https://drive.google.com/drive/folders/>, 2022.